

Intelligent Ground Vehicle Competition 2026

AutoNav & Self-Drive



Cornell Electric Vehicles

Cornell University

Design Report

[Vehicle Name]

[Vehicle Photo / Sketch]

[Faculty Advisor's Signed Statement of Integrity]

Team Captain

Cam Mazzacane

clm357@cornell.edu · (585) 474-5070

Key Team Member

[Name]

[email@cornell.edu] · [Phone]

Faculty Advisor

[Name]

[email@cornell.edu] · [Phone]

[Author Name]

[Author Name]

[Author Name]

[Author Name]

[Author Name]

[Author Name]

[Author Name]

Contents

1	System Requirements and Design	1
2	Mechanical Design	1
3	Safety	1
4	Electrical/Electronic Design	1
5	Perception	2
6	Driving Logic	5
7	Key Performance Indicators	5
8	Analysis of Complete Vehicle	5
9	Cyber Security Analysis	6

1. System Requirements and Design

Things to Write

- Describe the system engineering process you followed to identify important System and Subsystem requirements.
- List at least 2 requirements for: (a) the mechanical design of your vehicle, (b) safety, (c) the electrical/electronic components
- For each challenge you are targeting (AutoNav and SelfDrive), list at least two requirements that guided your design for each of: (a) perception, (b) driving logic, and (c) key performance indicators.
- For each requirement you listed (12 for one course, 18 if both) explain what in these rules elsewhere that drives this requirement, describe how you will quantifiably measure the requirement, and set a target value.

2. Mechanical Design

Things to Write

- Describe the mechanical design of your robot, including CAD or other drawings of your vehicle
- Describe significant decisions on the frame structure, housing, and/or overall design
- Describe significant components of your vehicle's drive system, electrical power system, and the control system that enables computer-controlled motion. Components acquired ready-made must be identified, but their internal components need not be described in detail.
- Describe the significant mechanical components, including the frame, structure, suspension, housing, and weather proofing. Clearly identify the aspects of your mechanical design that are driven by each mechanical system requirement you listed.
- For each mechanical requirement you

listed, compare the target value with the most recently measured actual value, and discuss what this tells you about your vehicle

3. Safety

Things to Write

- Describe the robot's safety aspects while being transported, parked, and charging.
- Describe the robot's safety aspects while operating on the course(s), including the mechanical and wireless ESTOP systems.
- Clearly identify the aspects of your safety efforts that are driven by each safety requirement you listed.
- For each safety requirement you listed, compare the target value with the most recently measured actual value, and discuss what this tells you about your vehicle

4. Electrical/Electronic Design

Things to Write

- Describe significant power, electrical, and electronic components in your vehicle, including electronics, computers, sensors, and motor controllers
- Describe significant decisions made in designing/selecting the electrical and electronic components
- Describe the power distribution system's capacity, max. run time, recharge time, and safety
- Clearly identify the aspects of your electrical and electronic components that are driven by each electrical/electronic components system requirement you listed.
- For each electrical/electronic components system requirement you listed, compare the target value with the most recently measured actual value, and discuss what this tells you about your vehicle

Things to Write

- Describe significant power, electrical, and electronic components in your vehicle, including electronics, computers, sensors, and motor controllers
- Describe significant decisions made in designing/selecting the electrical and electronic components
- Describe the power distribution system's capacity, max. run time, recharge time, and safety
- Clearly identify the aspects of your electrical and electronic components that are driven by each electrical/electronic components system requirement you listed.
- For each electrical/electronic components system requirement you listed, compare the target value with the most recently measured actual value, and discuss what this tells you about your vehicle

5. Perception

Things to Write

- If you are targeting both challenges, you must include separate discussions of how your perception system matches each course.
- Describe processing of the sensor data to identify course features and items around the vehicle, including obstacles and lane markings, and the use of machine vision to perceive the environment.
- If targeting AutoNav, describe how the system identifies lanes and obstacles.
- If targeting SelfDrive, describe how the vehicle senses pedestrians, stop signs, and road obstacles.
- Describe the internal representation of the course built from the perception data which is used by the driving logic. This might be a sophisticated internal representation of the environment or a simple list of features. Include a discussion

of how it is created and updated as the robot moves.

- Clearly identify how your design of the sensor data processing system was driven by the perception requirements you listed.
- For each perception requirement you listed, compare the target value with the most recently measured actual value, and discuss what this tells you about your vehicle
- Developed a custom C++ Iterative Closest Point (ICP) node to process LiDAR point clouds for robust spatial mapping.
- Implemented a 3D-to-2D vertical slicing filter (using ROS 2 pointcloud iterators) to compress 3D LiDAR data, isolating course obstacles at specific Z-heights while filtering out ground and ceiling noise to prevent false obstacle detection.

Lane Segmentation Architecture

- **ZED2 Hardware and Spatial Sensor Fusion:** The pipeline begins at the hardware level with the ZED2 stereo camera, which serves as our primary optical and spatial sensor. The camera features a built-in Inertial Measurement Unit (IMU), which the onboard SDK utilizes to continuously calculate its own internal Extended Kalman Filter (EKF). This hardware-level sensor fusion delivers a highly stable and drift-corrected positional pose, ensuring that our generated 3D point cloud remains consistent in physical space regardless of vehicle vibrations or rapid frame-to-frame displacements.
- **Traditional CV vs. Deep Learning:** Initially, we attempted a classical computer vision approach using Canny Edge Detection coupled with a Hough Transform. While this method was computationally inexpensive, it proved extremely brittle in unpredictable outdoor environ-

ments. Shadows, sun glare, and degraded paint easily broke the edge detection thresholds, and the Hough Transform frequently misidentified the edges of cones, barrels, or grass boundaries as lane lines. We pivoted to a Machine Learning approach because convolutional neural networks learn semantic context, allowing them to accurately segment drivable paths and lanes even when the physical lines are partially obscured or subjected to harsh lighting gradients.

- **Model Deep Dive (LFD_Roadseg):** For our core lane detection, we utilized the LFD_Roadseg architecture, which employs a dual-stream design to balance high-speed inference with accurate spatial resolution. As shown in Figure 1, the model processes two inputs simultaneously: a high-resolution image ($H \times W$) to preserve fine, pixel-level details like thin lane boundaries, and an aggressively downsampled low-resolution image to quickly capture the broader, global context of the road.

Both streams pass through independent feature extractors. The low-resolution contextual features (F_2^l) are processed by an Aggregation Module and upsampled (F_a^l) to match the high-resolution dimensions. The core of the network is the **Selective Fusion** module. It concatenates the high-res and upsampled low-res features, passing them through convolutional blocks and a Sigmoid function to generate a spatial attention mask. This mask is element-wise multiplied with the high-resolution features (F_1^h) to aggressively suppress background noise and highlight the drivable path, followed by a residual addition to preserve structural integrity.

Because this classifier outputs a single binary mask rather than distinct

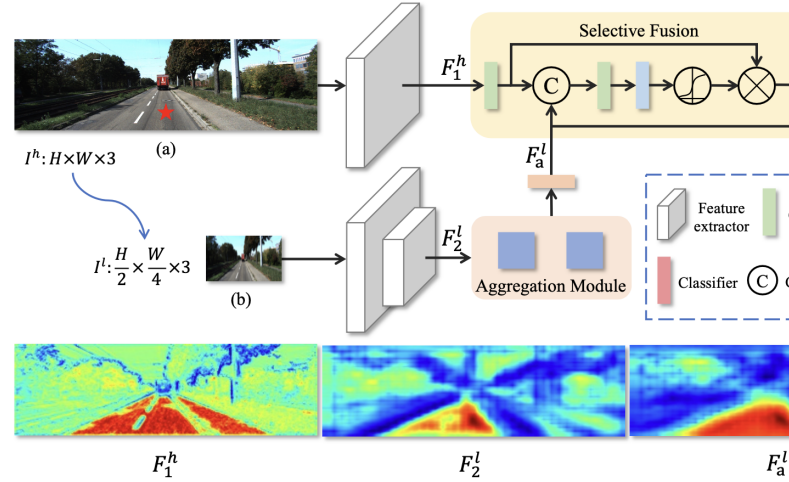


Figure 1: Architecture overview of the LFD_Roadseg neural network, detailing the dual-stream feature extraction and Selective Fusion module.

lane instances, we engineered a post-processing step. We apply a tensor squeezing operation to isolate the target dimensions before running a sliding-window histogram algorithm. This spatial histogram calculates pixel density peaks along the horizontal axis to track individual lane lines. When encountering intersections where lanes flare out and standard polynomial regression fails, the system triggers a custom radial fitting algorithm. Finally, we query the ZED2's synchronized spatial point cloud at those 2D coordinates to extract the third geometric dimension, naturally elevating our 2D classification into a 3D trajectory.

- **TensorRT Graph Optimization:** With the model architecture established, the primary bottleneck became inference latency. Running raw PyTorch models on the Jetson was completely unfeasible for real-time control loops. To achieve true deterministic speed, we compiled our trained LFD_Roadseg model into a highly optimized TensorRT engine. Ten-

TensorRT executed aggressive graph optimizations, fusing redundant layers, eliminating unused memory allocations, and quantizing our model weights from standard FP32 precision down to FP16. This unlocked massive parallel execution on the Jetson's Tensor Cores, dropping our inference latency significantly.

- **VRAM Pipeline and Serialization Zero-Copy:** Moving to TensorRT introduced integration challenges regarding data serialization within our Python stack. TensorRT engines demand raw, contiguous memory buffers mapped precisely to the GPU's hardware layout. To bridge this gap, we engineered a high-performance Python inference wrapper that keeps the entire data pipeline resident in video memory (VRAM). Instead of allowing the CPU to slowly serialize data back and forth between host and device memory, our pipeline maps the incoming ZED2 camera buffers directly into CUDA graphics memory, runs the TensorRT execution context in-place, and processes the squeezed tensors completely within the VRAM, bypassing the Python Global Interpreter Lock (GIL) entirely.
- **Environment Optimization via Pixi:** Our final deployment challenge was environment reproducibility. We originally used Docker, but the container overhead, massive image sizes, and clunky GPU passthrough configurations created immense friction when deploying directly on the vehicle's Jetson platform. We overhauled our environment by porting the system to Pixi. Pixi allowed us to explicitly lock our Python dependencies, CUDA toolkits, and hardware libraries in a single, lightweight Conda-based environment file, granting us native-speed execution and slashing our compilation times down to seconds.

Personal Reflection & Future Outlook

- **Theory vs. Physical Hardware Constraints:** Looking back at this semester, the biggest reality check for me was realizing just how brutal deploying to physical hardware actually is. It's one thing to get a model running smoothly in a Jupyter Notebook on my laptop, but getting it to survive the compute constraints of the Jetson while keeping the frame rate high enough to actually steer the car was a massive headache. The math has to actually translate to the metal. Building the model was only half the battle; fighting through serialization bottlenecks and memory management to make it fast took just as much engineering.
- **Algorithmic Improvements:** For next semester, I honestly want to try finding a model that natively splits the lanes within the road for us right out of the box. Doing all this tensor squeezing and sliding-window histogram math as a post-processing workaround is costing us speed, and it absolutely falls apart at intersections. We need to find or train something more robust to handle those flaring lane lines without relying on hacky radial fitting fail-safes.
- **Hardware and Sensor Architecture:** Also, we are leaving a lot of efficiency on the table. Right now, we rely entirely on the ZED2's internal IMU to calculate the EKF for spatial tracking. But the camera is bolted to the car, and the car is already computing its own EKF for the main drive system. So next semester, I want to change it to just rely on the car's EKF instead. If we stop doing the exact same math twice, then we could also put the ZED1 (has no IMU) camera on the car to maybe get a wider FOV, helping more with tight corners.

6. Driving Logic

Things to Write

- If you are targeting both challenges, you must include separate discussions of how your driving logic matches each course.
- Design of the lane following and obstacle avoidance systems must be specifically described, along with how the vehicle navigates through its environment. Describe how the system uses GPS for waypoint navigation and localization.
- If targeting AutoNav, describe how your robot uses the perception system's output and other information to follow lanes, detect and avoid obstacles, climb the ramp, and navigate to the GPS waypoints. Include how vehicle motion is generated and monitored to move the vehicle through the course, including how it deals with complex obstacles including switchbacks and center islands dead ends, traps, and potholes.
- If targeting SelfDrive, describe how the vehicle uses the perception system's output and other information to follow the rules of the road while navigating the course that includes pedestrians, stop signs, road obstacles, and other items.
- Clearly identify how your design was driven by the driving logic requirements you listed.
- For each driving logic requirement you listed, compare the target value with the most recently measured actual value, and discuss what this tells you about your vehicle

7. Key Performance Indicators

Things to Write

- If you are targeting both challenges, you must include separate discussions for each course.
- Describe how you will measure each of

the key performance indicators you selected to check that your robot is competitive on the target course(s), what value/result you are targeting, and how you selected that value/result for the KPI.

- For each KPI requirement you listed, compare the target value with the most recently measured actual value, and discuss what this tells you about your vehicle

8. Analysis of Complete Vehicle

Things to Write

- Describe lessons learned during construction and system integration
- Identify key mechanical and electrical components that have failed during testing and discuss what changes you made to prevent repeat failures. Did your changes prevent repeat failures?
- Describe your software testing, bug tracking, and version control processes
- Describe your simulation-based testing process (e.g., Gazebo, etc.)
- Describe physical testing to date (mechanical, electronic, indoors, real world, etc.) and identify key differences in actual performance compared to predictions from simulation.

Things to Write

- Describe lessons learned during construction and system integration
- Identify key mechanical and electrical components that have failed during testing and discuss what changes you made to prevent repeat failures. Did your changes prevent repeat failures?
- Describe your software testing, bug tracking, and version control processes
- Describe your simulation-based testing process (e.g., Gazebo, etc.)
- Describe physical testing to date (mechanical, electronic, indoors, real world, etc.) and identify key differences in ac-

tual performance compared to predictions from simulation.

9. Cyber Security Analysis

Things to Write

- Modern robotic and human driven vehicles are network-connected, contain large amounts of software from multiple vendors, and are an inviting target for cyber-attacks.
- List 3 cyber vulnerabilities present in your vehicle and discuss what steps should be taken to harden/remove them before it enters series production and is widely deployed.